UNIVERSIDAD DE CARABOBO
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación
Unidad Académica de Algoritmos y Programación
CAO402: Algoritmos II

Profesor: Álvaro Espinoza

Fecha: 25/06/2025

PARCIAL I: PROGRAMACIÓN ORIENTADA A OBJETOS
Período Lectivo 1-2025

Nombre y Apellido: _____ Cedula de Identidad:_____

PARTE I: Lea cuidadosamente la pregunta y seleccione la opción correcta. Se aplicará factor de corrección: 3 respuestas incorrectas eliminarán una correcta, una respuesta en blanco no afectará a las respuestas correctas (1 punto c/u)

a) Dada una clase A que contiene en agregación a una clase B. Y una clase C que hereda de A y B:
   a) Esta relación no es posible en la Programación Orientada a Objetos
   b) A tiene acceso a los métodos privados de B
   c) A solo puede contener elementos de tipo B
   d) C puede contener objetos de tipo C
   e) Las materias se pelean hasta el final

b) Sea una clase A que contiene a una clase B. A su vez, B contiene a una clase C:
   a) Por transitividad, A puede contener elementos de tipo C
   b) Es posible que el profesor quiera confundirnos
   c) A sólo puede contener objetos de tipo B.
   d) La clase C debe estar definida antes que la clase A para que el código compile
   e) A puede contener elementos de tipo C si C es pública

c) Dada una clase X con cinco (5) constructores polimórficos no heredados:
   a) Esto es una agresión
   b) Esto es posible gracias al polimorfismo dinámico
   c) Esto es posible gracias al polimorfismo estático
   d) Esto es un ejemplo de sobreescritura de métodos
   e) La mala definición de estos constructores es detectada en tiempo de ejecución

d) Sea A una clase abstracta con diez clases hijas.
   a) Solo una de las diez clases hijas puede implementar los métodos abstractos de A
   b) Las clases hijas están obligadas a heredar solo de A, ya que no se permite herencia múltiple con clases abstractas
   c) Si una clase hija no implementa todos los métodos abstractos de A, el programa no compila
   d) No tiene sentido instanciar la clase A al ser abstracta
   e) El profesor disminuirá la población de la materia a la mitad

e) Dada una clase W con una asociación "usa" en dirección a una clase A. Y A está compuesta por una clase B.
   a) Yo se programar. Me falta solo el título.
   b) La asociación "usa" permite a W acceder a los atributos privados de B
   c) W y B no están relacionados
   d) W y B están relacionados
   e) La composición entre A y B es más eficiente sustituirla por una herencia

f) Usted ha diseñado una clase A con todos sus atributos privados. Y se encuentra programando las clases B, C y D que heredan todas de A. Un colega le sugiere cambiar sus atributos a protegidos. ¿Cuál es la razón de esto?
   a) El polimorfismo no permitirá sobreescribir los métodos getters/setters de A en las clases hijas si sus atributos son privados
   b) Al profesor no le gusta que hagan eso
   c) Esto facilitará el trabajo con los atributos de A en las clases hijas ya que al ser protegidos se puede acceder a ellos directamente
   d) Cambiar los atributos a protegidos permitirá que las instancias de B, C y D accedan entre sí a sus atributos internos.
   e) Los atributos protegidos hacen que los objetos de las clases hijas se comporten como si fueran del tipo padre.

g) Usted está diseñando una clase Pedido, la cual necesita trabajar con objetos de la clase Producto. Un colega le pregunta qué tipo de relación establecer entre ambas clases. ¿Cuál de las siguientes afirmaciones representa mejor una asociación adecuada entre Pedido y Producto?
    a) Debería usar una relación de herencia, ya que un producto es un tipo de pedido.
    b) Una composición es adecuada si Producto debe seguir existiendo aunque se elimine el Pedido.
    c) Si Pedido tiene una lista de productos, eso obliga a usar composición, ya que hay múltiples instancias.
    d) Una asociación de uso es suficiente si Pedido solo necesita acceder a Producto en un método temporalmente
    e) La agregación no es válida porque ambas clases no pueden ser instanciadas por separado

h) Usted ha diseñado una jerarquía de clases donde: La clase Animal define comportamientos generales. La clase Mamifero hereda de Animal. La clase Perro hereda de Mamifero ¿Cuál de las siguientes afirmaciones es correcta respecto al uso de herencia en múltiples niveles?
    a) La clase Perro no puede sobrescribir métodos definidos en Animal porque no hereda directamente de ella
    b) Es preferible que Perro herede directamente de Animal para evitar depender de clases intermedias.
    c) Perro hereda todos los atributos y métodos públicos y protegidos de Animal a través de Mamifero.
    d) Mamifero debe ser abstracta para que Perro pueda heredar de ella correctamente.
    e) No debo rendirme hasta aprobar la materia

i) Una desarrolladora está implementando una clase Calculadora que tiene varios métodos llamados sumar(), cada uno con diferente cantidad o tipo de parámetros. El compilador decide automáticamente cuál método usar según los argumentos que se le pasan al invocar sumar(). ¿Cuál de las siguientes afirmaciones describe correctamente esta situación?
    a) Thanos tenía razón
    b) Se trata de polimorfismo dinámico porque el método sumar puede comportarse distinto en tiempo de ejecución.
    c) Esto es una sobrecarga de métodos, un caso típico de polimorfismo estático
    d) El comportamiento depende de qué clase hereda de Calculadora, por lo tanto es herencia múltiple
    e) No hay polimorfismo, ya que todos los métodos tienen el mismo nombre

j) Un estudiante está revisando dos fragmentos distintos de un sistema: En uno, hay una clase Calculadora con varios métodos sumar() que reciben distinta cantidad o tipo de parámetros. En otro, una variable de tipo Figura puede hacer llamadas a dibujar(), y dependiendo de si se trata de un Círculo o un Rectángulo, el comportamiento es distinto en tiempo de ejecución. ¿Cuál de las siguientes afirmaciones describe correctamente ambos casos?
    a) Honestamente creo que me gusta mas el calculo
    b) En ambos casos se aplica polimorfismo estático, ya que los métodos comparten nombre.
    c) El primer caso representa encapsulamiento, y el segundo, herencia
    d) El primer caso es polimorfismo estático (sobrecarga) y el segundo, polimorfismo dinámico (sobreescritura)
    e) Ninguno de los dos casos representa polimorfismo, ya que el comportamiento está completamente definido por el programador

PARTE II: Modelado de Clases: Realice el diagrama de clases correspondiente a la siguiente situación **(5 pts)**:

El Festival del Metal Eterno

El comité organizador de un festival de heavy metal extremo desea llevar el control de los asistentes y eventos del festival. De cada asistente se conoce su nombre, número de identificación, fecha de nacimiento, edad y fecha de registro. Se sabe que un asistente puede participar en diferentes actividades dentro del festival: músicos, staff técnico, fanáticos autoetiquetados, fanáticos de múltiples bandas o de una sola banda. Cabe destacar que una persona puede participar en más de una actividad, y que cada rol tiene información diferente asociada, según su tipo.

Se entiende que los fanáticos de múltiples bandas siguen a dos (2) o más bandas, lo cual los diferencia de los fanáticos de una sola banda, y también de los fanáticos autoetiquetados, quienes solo siguen su propia banda o proyecto musical. Por otra parte, se tiene información de los conciertos organizados por el staff técnico, los cuales pueden ser organizados por uno o varios miembros del staff y además, de cada concierto se conoce la fecha y hora exacta en que se realiza. Por supuesto, cada asistente asiste a los conciertos que más le interesan.

Se desea conocer siempre a qué concierto asiste cada persona y qué bandas son seguidas por los asistentes. Además, se requiere llevar un registro constante del número de personas registradas en el festival.

PARTE III: Implementación (5 pts):

Obras Teatrales

Se necesita modelar el sistema de organización interna de un teatro que gestione múltiples funciones de diferentes obras. Para ello, deberás diseñar un conjunto de clases que representen las **obras teatrales**, sus distintas **funciones** y los **actores** involucrados.

Requisitos del sistema:

1. Una **obra teatral** debe:
    - Tener un título y un género (por ejemplo: "Comedia", "Drama", "Tragedia").
    - Contener una lista de funciones que se han programado para esa obra.
    - Poder devolver el total de funciones registradas y la lista de todos los actores que han participado en alguna función de la obra (sin repeticiones).
2. Cada **función** representa una única presentación en una fecha determinada, e incluye:
    - La fecha (como cadena o número).
    - Una referencia a la obra a la que pertenece.
    - Una lista de actores asignados a esa función.
3. Cada **actor** debe tener:
    - Nombre completo.
    - Experiencia (expresada como número de años en el teatro).

Indicaciones adicionales:

- Una función siempre pertenece a una obra, y cada función está conformada por varios actores. Una obra existe por sus funciones.
- Es obligatorio declarar los métodos *getters* y *setters* para todos los atributos principales. No es necesario implementar su lógica interna, pero deben figurar en el código fuente.
- El modelo debe permitir gestionar el sistema desde la clase "Obra Teatral", sin estructuras externas auxiliares.

……

Sobró espacio, te regalo un código para que te recuerdes de estudiar arquitectura :)

```
.data
mensaje: .asciiz "Te recuerdo que debes estudiar tambien arquitectura :)\n"
.text
.globl main
main:
    li $v0, 4            # Código de syscall para imprimir cadena
    la $a0, mensaje      # Cargar dirección del mensaje en $a0
    syscall              # Llamada al sistema
    li $v0, 10           # Código de syscall para salir del programa
    syscall              # Salir
```

“"Los algoritmos son la poesía de la computación."” *- Donald Knuth*